Existing Upload + Device-Update Mechanisms (grounded inventory)

Revision: 1 **Last modified:** 2026-07-10T11:06:45Z

> READ-ONLY codebase discovery for the multi-account feature. Every claim below cites `file:line`. Where a mechanism does NOT exist, that is stated as a fact with the searches performed — never a guess (§11.4.6). Scope: what EXISTS today, so a to-be plan can EXTEND it rather than reinvent it.

One-paragraph bottom line. Mechanism **(A) upload/publish** EXISTS on the server as a real, validated multipart pipeline (`POST /artifacts/upload` → `POST /releases` → `POST /deployments`) and on the project side as an operator **web SPA** (`clients/ota-manager`, React + Tauri) that drives those endpoints — but it is **single-tenant**: nothing about upload/release/deployment/device is scoped to a project or account today. Mechanism **(B) device-side update client** EXISTS as the `ota-android-agent` submodule (a headless WorkManager **poll** worker) plus the server's `GET /client/update` endpoint — the device identifies itself **only** by its bearer-token subject (`deviceId`), with no account/project. There is **no** notification/push channel (poll-only) and **no** device-side "new version" setup/consent wizard (the only "wizard" is the operator-side create-release dialog). A `Project` + per-project-ACL model is **declared** in the store but is **not wired** into the OTA data model, and the SPA's project switcher is a hardcoded mock.

---

1. UPLOAD / PUBLISH path (server side)

Route table (all under `cfg.APIBasePath`, default `/api/v1`)

`server/internal/api/server.go`: - `server.go:197` — `auth.POST("/artifacts/upload", requireRole(RoleOperator, RoleAdmin), s.handleUploadArtifact)` - `server.go:198` — `auth.GET("/

artifacts/:artifactId", requireRole(RoleViewer, RoleOperator, RoleAdmin), s.handleGetArtifact) (metadata only — see §Honest gaps) - server.go:205-207 — POST /releases, GET /releases, GET /releases/:releaseId - server.go:209-211 — POST /deployments, GET /deployments, GET /deployments/:deploymentId - server.go:215-217 — staged rollout create/get/evaluate - Auth wrapper: server.go:189-190 — auth := v1.Group(""); auth.Use(s.authMiddleware(), s.auditMiddleware()). Every write is bearer-token authenticated + audited.

Upload handler — request shape, auth, validation, where bytes land

server/internal/api/handlers_artifact.go: - handleUploadArtifact — handlers_artifact.go:47. Requires multipart/form-data (:51), enforces upload cap (:49, s.cfg.MaxUploadBytes). - Multipart parts (:78-144): file (payload bytes), metadata (JSON ArtifactUploadMetadata), optional sha256, optional signature. - Required metadata fields (:101-109): sha256, signature, version, os, target_model (wire struct ArtifactUploadMetadata in server/internal/api/wire.go; client mirror clients/ota-manager/src/types/api.ts:723-735). - Validation pipeline S1–S6: - S1 structure (must be ZIP, payload.bin STORED) — handlers_artifact.go:111-123, validateStructure :253-272. - S3 trusted key resolved from server config ONLY — :126 → resolvePublicKey :283-288 (see §3). - S3 detached signature resolved — :134, resolveSignature :293-310. - S2 hash file resolved — :144, resolveHashFile :315-323. - S4 prior version for monotonicity read via LatestRelease(os, target_model) — :148 (keyed on OS+model, NOT project). - S2..S6 run in otavalidator.Validate(in) — :152-173 (brick github.com/HelixDevelopment/ota-artifact-validator). - **Accept path (where the record lands)** — :176-204: builds store.Artifact and calls s.repo.CreateArtifact. StorageRef is a **synthesized placeholder** fmt.Sprintf("s3://helix-artifacts/%s", artifactID) (:184) — the actual **payload BYTES are validated then discarded**; only metadata + the verified base64 signature (:196) + payload headers are persisted. Both store impls persist metadata only: server/internal/store/memory.go:198 and server/internal/store/postgres.go:209-226.

Release + deployment publish

- handleCreateRelease — server/internal/api/handlers_release.go:16. Referenced artifact must exist + be Verified (:43-52); version strictly-monotonic vs LatestRelease(os,

`target_model)` (`:56-69`); creates `store.Release` status `published` (`:71-86`).
- `handleCreateDeployment` — `server/internal/api/handlers_deployment.go:28`. MVP accepts only `strategy == "all-targets"` (`:39-43`); conflict if an active deployment already targets `(os, model, group)` (`:78-83`); stamps target version on matching devices (`:104`, `assignTargetVersion :162`).

Is upload scoped to a project already? NO.

- `store.Artifact` (`server/internal/store/store.go:58-75`), `store.Release` (`:77-88`), `store.Deployment` (`:90-99`), `store.Device` (`:35-56`) carry **no `ProjectID` / account field**.
- The `Repository` artifact/release/deployment/device methods take **no project argument** (`store.go:334-360`); device/deployment resolution keys on `(OS, target_model, group)` only.
- Token claims carry only `sub`, `roles`, `iat`, `exp` — **no account/project claim** (`server/internal/api/token.go:31-36`).
- A `Project` + `ProjectAccess` (per-project RBAC ACL) model DOES exist (`store.go:287-332`; handlers `server/internal/api/handlers_project.go`) but is a **separate, parallel construct not attached to the OTA data model** — see §7.

---

2. UPLOAD client / CLI (project side)

What exists: the operator web SPA `clients/ota-manager` (React + Tauri desktop shell)

- Upload mutation — `clients/ota-manager/src/hooks/use-artifacts.ts:35-51` (`useUploadArtifact`): builds a `FormData` with `file` + `metadata` (+ optional `sha256`/`signature`) and calls `apiMultipartPost<Artifact>('/artifacts/upload', formData)`.
- Transport wrapper — `clients/ota-manager/src/lib/api-client.ts:74-85` (`apiMultipartPost`). Base URL is same-origin `/api/v1` by default (`:20`).
- Auth — `api-client.ts:28-36`: an axios request interceptor attaches `Authorization: Bearer <token>` from the auth store. **No project/account header, query, or path segment is added on any request.**
- Publish "wizard" — the multi-step create-release flow: `docs/superpowers/specs/2026-06-19-ota-management-client-design.md:170` ("CreateReleaseDialog (multi-step wizard)"); adapter

shims `clients/ota-manager/src/hooks/useCreateRelease.ts:1` and `useArtifact.ts:1`. This is the OPERATOR publish wizard, not a device wizard (see §6).
- Serve/embed path — the SPA is embedded into the Go binary (`server/internal/api/embed.go`, `MountManagerUI` at `server.go:253`); build/embed procedure in `clients/ota-manager/server-integration.md`.

What is a stub, not real

- **Project switcher is a MOCK** — `clients/ota-manager/src/features/layout/project-switcher.tsx:13-19`: `MOCK_PROJECTS` hardcoded (`ATMOSphere`, `Helix OTA`) held in local `useState`; **no API call, no effect on any request**.
- **Projects hook is a stub** — `clients/ota-manager/src/hooks/use-projects.ts:21-45`: `useProjects` returns `placeholderData` of a single `"Default Project"` ("Single-project mode — all resources live here") and treats a 404 from `/projects` as expected.
- **No list-artifacts source** — `clients/ota-manager/src/hooks/useUploadArtifact.ts:4-7` documents the KNOWN GAP that the server exposes no list-artifacts endpoint, so the release picker's artifact list is empty.

Is there a project-side CLI / SDK to push updates? NO dedicated one.

- `server/cmd/` binaries: `ota-server` (the control plane), `ota-device-emu` (a DEVICE emulator, not an uploader — see §4), `applyport`. There is **no** `cli/` or `client/` upload tool and **no** publish SDK; the only publish surfaces are the SPA and raw REST.

Missing for a production "project authenticates to account+project and uploads"

- No account/project binding in auth (login/token — §7).
- No project scoping on artifact/release/deployment (§1).
- No real artifact object storage (bytes discarded; `StorageRef` placeholder — §1, §Honest gaps).
- No list-artifacts endpoint; no CLI/SDK; project switcher/hook are mocks.

---

3. Artifact signing / trust boundary

- **Trust boundary (verification key from server config ONLY):** `resolvePublicKey` — `server/internal/api/handlers_artifact.go:283-288`. The key comes solely from `s.pubKey`; the request can never supply a key (security comment `:274-282`; matches project CLAUDE.md's stated trust boundary).
- **Key source (single, GLOBAL):** `server/internal/config/config.go:72-77` `ArtifactPublicKey`, loaded base64 from env `HELIX_ARTIFACT_PUBKEY` (`config.go:187-192`); wired into the server at `server.go:109-112`. It is **one ed25519 key for the whole control plane** — not per-account/per-project (the seam for per-account key scoping — §7).
- **Verification:** S3 runs inside `otavalidator.Validate` with `Input.PublicKey = pubKey` (`handlers_artifact.go:153-169`).
- **Verified signature is the single source of truth stored + served:** the exact detached signature that S3 verified is persisted (`handlers_artifact.go:196`, `store.Artifact.Signature` `store.go:70-72`) and later handed to the device in the update offer (`handlers_client.go:81`).
- **Device re-verifies before apply:** the agent's `VerifyBeforeApply` gate re-checks SHA-256 + signature validity before any apply, ordered `MALFORMED_DIGEST → HASH_MISMATCH → SIGNATURE_INVALID` (`submodules/ota-android-agent/README.md:37`; `core/.../verify/VerifyBeforeApply.kt`), and a rejected artifact never reaches `update_engine` (`OtaPollWorker.kt:99-113`).

---

4. Device-side update client

Where it lives: `submodules/ota-android-agent` (headless WorkManager worker)

- `:core` = pure Kotlin/JVM logic; `:android` = WorkManager wiring (`README.md:13-47`).
- **Transport = POLL** (no push): `PollScheduler.schedule(...)` runs a 15-min periodic worker with jitter + exponential backoff (`README.md:46`, usage `:64-68`; source `android/.../poll/PollScheduler.kt`).
- The poll port calls the server: `ControlPlaneClient.pollForUpdate()` → "GET /api/v1/client/update -> 200 manifest | 204 no update | transient error" (`android/.../poll/Ports.kt:29-33`).

- One cycle = poll → download → **verify-before-apply** → apply → telemetry: `OtaPollWorker.runCycle` (`android/.../poll/OtaPollWorker.kt:59-126`). Verify-reject deletes the artifact and never applies (`:99-113`).

Server endpoint the device contacts

- `handleClientUpdate` — `server/internal/api/handlers_client.go:21`. Route `server.go:223` (`GET /client/update`, `requireRole(RoleDevice)`). Returns `204` when on-target, else `200` with `UpdateAvailable` (`handlers_client.go:70-101`).
- Active-deployment resolution keys on `(dev.OSType, dev.Model, dev.Group)` — `handlers_client.go:43` (no project/account).

What the device sends to identify itself

- **Device id comes from the bearer-token subject** — `handlers_client.go:22-23` (`deviceID := claims.Subject`). Optional `current_version` query short-circuit — `:38`.
- Agent DTO confirms the same: `UpdateCheckRequest { deviceId, currentVersion? }` where "the calling deviceId comes from the token `sub`" — `submodules/ota-android-agent/core/.../protocol/Dtos.kt:33-36`.
- **No project / account / org / tenant is ever transmitted** by the device.
- Device token is minted at registration: `mintDeviceToken` (role=`device`, sub=`deviceId`) — `handlers_device.go:157-159`; issued by `handleRegisterDevice` (`handlers_device.go:16-85`), which itself requires an **operator/admin** token (`server.go:192`). So enrollment is operator-driven; `store.Device` has no `ProjectID` (`store.go:35-56`).

Concrete HTTP client / device config

- In the submodule, `ControlPlaneClient`, `Downloader`, `Verifier`, `Telemetry` are **abstract interfaces** — "conceptually over the http3 transport / Auth-KMP / Security-KMP" — with no concrete impl and no server-URL/credential config bundled (`Ports.kt:1-6, 29-73`; README §Public API `:41-47`).
- The concrete reference device client that performs real login + register + poll + telemetry is the Go emulator: `server/internal/deviceemu` driven by `server/cmd/ota-device-emu/main.go` (purpose `:1-19`; flags `-base`, `-admin-user`/`-admin-pass` (operator login), `-hardware-id`, `-current-version`, `-loop`/`-interval` at `:36-51`). It

flashes only in emulation; login/register/poll/telemetry are real HTTP.

---

5. NOTIFICATION mechanism

There is none — the system is pull-only. - Devices learn of a new version by **polling** `GET /client/update` and receiving `200 UpdateAvailable` vs `204` (`handlers_client.go:21-101`; agent `OtaPollWorker.kt:59-69`). - Searches for a server→device / server→user push found nothing: no webhook, FCM/Firebase-messaging, WebSocket, push-notification, or OneSignal wiring in `server/internal`, `submodules/ota-android-agent`, or `submodules/ota-protocol` (the only grep hits were the word inside `server/internal/api/security_headers.go` comments and the bundled toast library in the built `manager-dist` JS — no push transport). - The only device→server flow is telemetry: `POST /client/telemetry` (`handlers_client.go:106`), and deployment progress is **derived server-side** from that telemetry (`handlers_deployment.go:202 deriveProgress`) — the operator UI observes progress by polling `GET /deployments/:id`.

---

6. SETUP WIZARD (device / consent)

No device-side setup/consent/"new version available" wizard exists. - The `ota-android-agent` is a **headless** WorkManager worker that **auto-applies** a verified package with no user-facing consent UI (`OtaPollWorker.kt:116-124`; README overview `:15-26`). `ota-update-engine-bridge` is likewise headless (apply/boot-state only: `EngineStatus.kt`, `ApplyRequest.kt`, `BootStateObserver.kt`). - Searches for `onboarding` / `first-run` / `enrollment` / `provisioning wizard` in the agent and SPA returned no UI (the single `wizard`-adjacent hit in the agent was a scheduling comment in `PollScheduler.kt`, not a UI). - The only "wizard" in the codebase is the **operator publish** flow — the multi-step `CreateReleaseDialog` in the SPA design (`docs/superpowers/specs/2026-06-19-ota-management-client-design.md:170`; shims `useCreateRelease.ts:1`, `useArtifact.ts:1`). It is a publish wizard, not a device "informs users of a new version" wizard.

---

7. Extension points for multi-account

Upload-CLI / publish path (project side)

Seam	Concrete `file:line`	Current readiness
Auth token claims (add `account`/`project`)	`server/internal/api/token.go:31-36` (`Claims{sub,roles,iat,exp}`)	present-but-prototype (static `UserDirectory`, HMAC-opaque token; `server.go:24-26`, `handlers_auth.go:77-98`)
Login → token issuance (bind account+project)	`server/internal/api/handlers_auth.go:77` (`handleLogin`), `:123` (`issueTokenPair`)	present-but-prototype
Artifact record (add `ProjectID`)	`handlers_artifact.go:176-198` (build `store.Artifact` + `CreateArtifact`); type `store.go:58-75`	absent (no project field)
Release / deployment scoping	`handlers_release.go:71-86`; `handlers_deployment.go:88-100`; types `store.go:77-99`	absent (no project field)
Per-project RBAC ACL (already built)	`store.go:287-332` (`Project`, `ProjectAccess`); `handlers_project.go:37-92` (`requireProjectAccess`, `isRoleAtLeast`)	present-but-prototype (memory-backed; **not enforced** on artifact/release/deployment routes — those routes carry no `:projectId`, `server.go:197-217`)
Signing-key scoping (per-account key registry)	`handlers_artifact.go:283-288` (`resolvePublicKey`); config `config.go:72-77` + `server.go:109-112`	present-but-GLOBAL (one `HELIX_ARTIFACT_PUBKEY`; needs a per-account/project key lookup)
SPA project selection + request scoping	`clients/ota-manager/src/features/layout/project-switcher.tsx:13-19` (mock); `src/hooks/use-projects.ts:21-45` (stub); `src/lib/api-client.ts:28-36` (no project header)	present-but-prototype / absent

Device-update client

Seam	Concrete `file:line`	Current readiness
Device registration (add `ProjectID`/ account)	`handlers_device.go:16-85` (`handleRegisterDevice`); type `store.Device` `store.go:35-56`	absent (no project field)
Device token claim (carry project/ account)	`handlers_device.go:157-159` (`mintDeviceToken`) → `token.go:61` (`Mint`)	absent
Update-check scoping	`handlers_client.go:43` (`ActiveDeploymentForTarget(os,model,group)`) — add project filter	absent
Agent identity DTO + device config	`submodules/ota-android-agent/core/.../protocol/Dtos.kt:33-36` (`UpdateCheckRequest` has no account); concrete `ControlPlaneClient` + server-URL/creds config not in submodule (`Ports.kt:29-33`)	absent (concrete client + config unimplemented)

Honest production-readiness ratings

- Upload endpoint + S1–S6 validation pipeline: **present** and robust, BUT the full publish is **present-but-prototype** because artifact **bytes are not stored** (placeholder `StorageRef`, `handlers_artifact.go:184`) and there is **no list-artifacts endpoint** (`useUploadArtifact.ts:4-7`).
- Project-side upload client (SPA): **present-but-prototype** (drives real REST; project switcher/hook are mocks; no CLI/SDK).
- Multi-tenant project model: **present-but-prototype** (ACL + CRUD exist; **not wired** into the OTA data model or the SPA data flow).
- Device update client (`ota-android-agent`): **present-but-prototype** (`:core` logic real + unit-tested; concrete HTTP client, device config, and real on-device Android build are not in-repo — see `submodules/ota-android-agent/BUILD_STATUS.md`).
- Notification / push: **absent** (poll-only by design).
- Device-side setup/consent wizard: **absent**.

Files read (provenance)

Server (Go, `server/internal/api/`): `server.go`, `handlers_artifact.go`, `handlers_client.go`, `handlers_auth.go`, `handlers_device.go`, `handlers_project.go`, `handlers_release.go`, `handlers_deployment.go`, `token.go`. Store: `server/internal/store/store.go` (interface + all domain types); grep of `memory.go`/`postgres.go` for `CreateArtifact`/`StorageRef`. Config: `server/internal/config/config.go` (grep for `ArtifactPublicKey`/`ArtifactBaseURL`/`MaxUploadBytes`/`HELIX_ARTIFACT_PUBKEY`). Server binaries: `server/cmd/ota-device-emu/main.go` (listed `server/cmd/{ota-server,ota-device-emu,applyport}`).

Client (`clients/ota-manager/`): `src/hooks/use-artifacts.ts`, `src/hooks/useUploadArtifact.ts`, `src/hooks/use-projects.ts`, `src/lib/api-client.ts`, `src/types/api.ts`, `src/features/layout/project-switcher.tsx`, `server-integration.md`; `src` tree listing.

Device agent (`submodules/ota-android-agent/`): `README.md`, `CLAUDE.md`, `android/.../poll/OtaPollWorker.kt`, `android/.../poll/Ports.kt`, `core/.../protocol/Dtos.kt`; file tree of agent + `ota-update-engine-bridge`.

Design/search: `docs/superpowers/specs/2026-06-19-ota-management-client-design.md` (wizard reference); repo-wide greps for `wizard`/`push`/`webhook`/`fcm`/`websocket`/`notification`/`onboarding`/`enrollment`; `docs/research/accounts/` directory listing (only `00_INDEX.md` present).

Honest gaps (undeterminable from code / explicitly not implemented) — §11.4.6

1. **Artifact byte storage is not implemented in this repo.** `handleUploadArtifact` validates the uploaded bytes then discards them; `StorageRef` is a synthesized placeholder string (`handlers_artifact.go:184`) and the device download URL is `ArtifactBaseURL/<id>.zip` (`handlers_client.go:232-234`) pointing at an external "Storage brick" that is not present here. Whether a real object store exists in deployment cannot be determined from this codebase.
2. **Concrete device HTTP client is not in the agent submodule.** The agent ports (`ControlPlaneClient`, `Downloader`, `Verifier`, `Telemetry`) are interfaces only (`Ports.kt`); the real Android impl (server-URL

config, token storage, credential source, account binding) is not in-repo. The Go `deviceemu` is the only concrete reference client.

3. **`Project` intent vs reality.** The `Project` type comment claims it is "a named container for devices, releases, and deployments, providing multi-tenant isolation" (`store.go:287-288`), but no artifact/release/deployment/device type or repository method references a project. The gap between the stated intent and the wired reality is the core of the multi-account work.
4. **No `account`/`organization` concept at all.** The codebase models `Project` (+ per-project role ACL) but there is no higher `account`/`org`/`tenant` entity anywhere in `store.go` or the API. A multi-account plan must decide whether "account" maps onto the existing `Project` or introduces a new parent entity.
5. **Whether the SPA/agent are meant to be the production clients** (vs the Go emulator being throwaway tooling) is a product decision not derivable from code; both SPA project scoping and the agent's concrete client are unfinished.